

Finny — техническая документация прототипа

Android-приложение с игровыми сценариями финансовой грамотности. Текущий идентификатор приложения: `com.example.myapplication`, отображаемое имя: `My Application`.

1. Назначение продукта и быстрый запуск

Назначение

Finny — однопользовательский Android-прототип для обучения финансовой грамотности через сюжет, диалоги и игровые ситуации. Игрок управляет персонажем, перемещается между локациями, изучает экономические термины, зарабатывает и тратит игровую валюту, использует банковские инструменты, покупает одежду и проходит мини-игры.

Основные учебные темы текущей версии: обязательные и необязательные траты, накопления, дебетовая карта и банковский счёт, накопительный счёт и вклад, процентная ставка, кредит, банк, НДС, НДС, сложный процент, пассивный доход и инфляция.

Быстрый запуск из Android Studio

1. Открыть корневую папку проекта `MyApplication` в Android Studio.
2. Дождаться Gradle Sync.
3. Установить Android SDK Platform 37, если IDE предложит это сделать.
4. Подключить устройство Android по USB с включённой USB debugging либо запустить эмулятор.
5. Выбрать конфигурацию `app` и нажать **Run**.

Быстрый запуск из консоли

Windows PowerShell:

```
.\gradlew.bat :app:assembleDebug
.\gradlew.bat :app:testDebugUnitTest
```

Linux/macOS:

```
chmod +x ./gradlew
./gradlew :app:assembleDebug
./gradlew :app:testDebugUnitTest
```

Debug APK после успешной сборки: `app/build/outputs/apk/debug/app-debug.apk`.

2. Состав репозитория

```
MyApplication/
├── app/
│   ├── build.gradle.kts
│   └── src/
│       └── main/
│           ├── AndroidManifest.xml
│           └── java/
│               └── com/example/myapplication/
```

```
├── MainActivity.kt
├── back/           # состояние и локальное сохранение
├── Dialogs/        # логика разблокировки учебных диалогов
├── front/          # локации, HUD, карта, пролог
├── market/         # каталог и UI магазина
├── wardrobe/       # одежда и отрисовка персонажа
├── ru/finny/games/
│   ├── api/        # общий API мини-игр
│   ├── bikes/      # велогонка
│   ├── grocery/    # продуктовая мини-игра
│   ├── warehouse/  # головоломка склада
│   └── res/         # изображения, темы, строки
├── test/           # JVM unit-тесты
├── androidTest/    # instrumented Compose-тесты
├── Clothes/        # исходные изображения одежды
├── room/           # исходные изображения комнаты
├── gradle/         # wrapper и JVM criteria
├── build.gradle.kts
├── settings.gradle.kts
└── INTEGRATION.md  # заметки по интеграции модулей
```

Ключевые точки входа:

- MainActivity.kt — корневая Compose-функция GameApp, маршрутизация локаций, магазина, шкафа и мини-игр.
- data_storage.kt — GameState, Game_progress, GameDataStore.
- PlayerSave.kt — атомарное сохранение кошелька, банковских значений, инвентаря и экипировки в SharedPreferences.
- MarketCatalog.kt — редактируемый каталог магазина.
- ClothingCatalog.kt — связь одежды с ресурсами и слотами персонажа.
- MiniGameApi.kt — единый формат результата мини-игр и базовая политика наград.

3. Требования к окружению и сборка release APK

Зафиксированные версии

Компонент	Версия / значение
Android Gradle Plugin	9.3.2
Gradle Wrapper	9.5.0
Kotlin Compose plugin	2.2.10
Gradle daemon JVM	JDK 25
Java source/target compatibility	Java 11
compileSdk	Android API 37
targetSdk	Android API 37
minSdk	Android API 24
Compose BOM	2026.02.01
AndroidX Activity Compose	1.8.0
AndroidX Lifecycle Runtime KTX	2.6.1
AndroidX Core KTX	1.10.1

Компонент	Версия / значение
DataStore Preferences	1.1.1
JUnit	4.13.2
AndroidX Test JUnit	1.3.0
Espresso	3.7.0

JDK 25 задан в `gradle/gradle-daemon-jvm.properties`. Gradle настроен на Foojay resolver, поэтому совместимый JDK может быть автоматически разрешён/загружен при наличии доступа к сети. Android SDK задаётся локально через `local.properties`; этот файл не должен коммититься.

Проверка окружения

```
.\gradlew.bat --version
```

В выводе необходимо проверить Gradle 9.5.0 и JVM, подходящую под критерий JDK 25.

Сборка unsigned release APK

1. Выполнить unit-тесты:

```
.\gradlew.bat :app:testDebugUnitTest
```

2. Собрать release-вариант:

```
.\gradlew.bat :app:assembleRelease
```

3. В текущем `app/build.gradle.kts` нет `signingConfig` для release. Поэтому командная сборка создаёт неподписанный артефакт, обычно:

```
app/build/outputs/apk/release/app-release-unsigned.apk
```

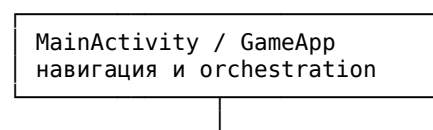
Сборка подписанного release APK через Android Studio

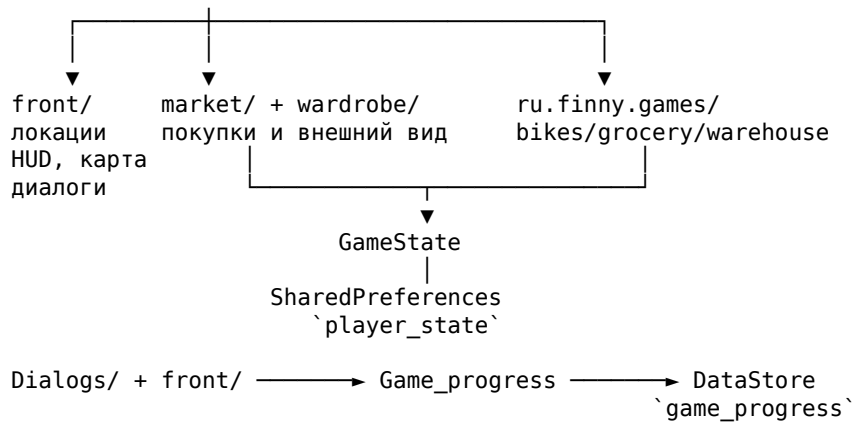
1. **Build → Generate Signed App Bundle or APK.**
2. Выбрать **APK**.
3. Выбрать модуль `app`.
4. Выбрать существующий keystore либо создать новый.
5. Указать alias и пароли ключа.
6. Выбрать `build variant release`.
7. Включить подписи схемами, предложенными Android Studio.
8. Завершить сборку и взять путь к APK из Event Log.
9. Установить APK на тестовое устройство и выполнить smoke-test из раздела 10.

Keystore и пароли запрещено добавлять в Git. Для CI рекомендуется передавать их через secrets и Gradle properties окружения.

4. Функциональная и компонентная архитектура

Общая схема





Компоненты

Компонент	Ответственность
MainActivity.kt / GameApp	Жизненный цикл приложения, загрузка прогресса, маршрутизация локаций, запуск/завершение мини-игр, показ модальных окон.
front/	Игровые локации, пролог, верхние показатели, нижнее меню, миникарта, термины/квесты, банковское окно.
Dialogs/	Правила разблокировки терминов и следующих диалогов.
back/GameState	Баланс, показатели персонажа, банковские остатки, покупки, экипировка, расчёт итогов мини-игр.
back/GameDataStore	Сохранение пролога, терминов, диалогов, квестов и старого инвентаря в Preferences DataStore.
back/PlayerSave	Сохранение текущего кошелька/банка/инвентаря/экипировки в SharedPreferences.
market/	Data-driven каталог, карточки товара и проверка покупки.
wardrobe/	Слоты TOP/PANTS/SHOES, наложение спрайтов одежды на персонажа.
games/api	Контракт MiniGameResult, защита активной сессии, общая политика награды.
games/warehouse	BlockBlast-подобная головоломка склада.
games/bikes	Велогонка с местом, столкновениями и денежной наградой.
games/grocery	Сбор продуктовой корзины с бюджетом, ценами и испорченными товарами.

Поток данных

1. Пользовательское действие меняет GameState либо Game_progress.
2. GameState сразу сохраняет критичные экономические данные через PlayerSave.
3. Учебный прогресс сохраняется через GameDataStore.saveProgress().
4. Compose наблюдает состояние и перерисовывает UI.
5. Мини-игра возвращает MiniGameResult; GameApp передаёт его в GameState.settleMiniGame().
6. Для продуктовой мини-игры деньги списываются сразу при подборе товара, а итоговый callback повторно сумму не списывает.

Серверная часть и OpenAPI

Серверной части в текущем репозитории нет. Сетевые разрешения не объявлены, API-клиент отсутствует, синхронизация с удалённым хранилищем не выполняется. Поэтому схема server deployment и OpenAPI для текущей версии **не применимы**.

5. Структура данных профиля, экономики, заданий и прогресса

Профиль

Отдельная сущность пользовательского профиля (id, имя, возраст, e-mail и т. п.) не реализована. Приложение хранит только локальное игровое состояние. В прологе используется текстовый placeholder ИМЯ, но он не связан с сохраняемым профилем.

GameState — текущее состояние экономики и персонажа

Поле	Начальное значение	Ограничения / назначение	Хранилище
money	1000	не меньше 0; общий кошелёк	player_state
food	20	0..100; сытость	player_state
weariness	100	0..100; фактически используется как положительная шкала бодрости, несмотря на имя поля	player_state
savingsBalance	0	накопительный счёт	player_state
depositBalance	0	сумма вклада	player_state
depositDaysLeft	0	при открытии вклада устанавливается 2	player_state
depositActive	false	признак активного вклада	player_state
ownedProductIds	∅	ID купленных товаров	player_state
equippedProductIds	∅	максимум одна вещь на слот одежды	player_state
lastSettledSession	null	защита от двойного начисления награды	player_state
inventoryMigrated	false	однократная миграция старого инвентаря из DataStore	player_state

Game_progress — образовательный и сюжетный прогресс

Содержит:

- prolog_completed — завершён ли пролог;
- progress_terms — карта термин → разблокирован;
- progress_quests — карта задание → выполнено;
- карты доступности диалогов для отца, матери, сотрудника банка, школы, склада и парка;

- устаревшие `progress_items_owned` и `progress_equipped`, используемые только как источник миграции старых сохранений.

В текущем списке заданий есть четыре записи:

- сходить на работу и познакомиться с НДФЛ;
- воспользоваться накопительным счётом;
- воспользоваться вкладом;
- сходить в магазин и познакомиться с НДС.

Фактическая автоматическая логика завершения этих квестов в текущем коде не подключена: `unlockQuest` во всех реализованных диалогах пуст.

Каталог магазина

`MarketCatalog` имеет иерархию:

`Department` → `Category` → `Group` → `MarketProduct`

`MarketProduct`:

```
id: String           // стабильный ID, участвует в сохранении
title: String
price: Int
description: String
imageRes: Int?
placeholderColor: Long
```

Существующие отделы: одежда, дом, спорт, техника. Для одежды есть отдельное соответствие `productId` → `ClothingSlot` + `imageRes` + `crop` + `placement`.

Состояние мини-игр

Долговременно сохраняется только итог, влияющий на общий кошелёк/покупки. Активная симуляция гонки и продуктовой игры удерживается в `MiniGameMemory` при пересоздании `Activity`. После полного уничтожения процесса незавершённая симуляция закрывается.

6. Матрица соответствия функциональным требованиям

Официальный внешний перечень функциональных требований с его исходными ID в архиве отсутствует. Поэтому матрица ниже фиксирует фактические функции текущего прототипа и привязывает их к коду/тестам. Если в конкурсном ТЗ есть отдельные ID, их следует перенести в первый столбец без изменения ссылок.

ID	Требование	Статус	Реализация / проверка
FR-01	Первый запуск с сюжетным прологом и сохранением завершения	Готово	<code>front/prolog.kt</code> , <code>PrologTest.kt</code>
FR-02	Навигация по дому и миникарте	Частично	<code>MainActivity.kt</code> , <code>front/indicate.kt</code> , <code>GameIntegrationTest.kt</code> . Переход в школу сейчас не подключён в <code>GameApp</code> .

ID	Требование	Статус	Реализация / проверка
FR-03	Словарь финансовых терминов и постепенная разблокировка	Готово	TERMS_DEFINITIONS в front/indicate.kt, Dialogs/dialogs.kt
FR-04	Система заданий	Частично	Структура и UI есть в data_storage.kt и front/indicate.kt, но автоматическое завершение квестов не подключено.
FR-05	Общий кошелёк и защита от отрицательного баланса	Готово	GameState, PlayerStateTest.kt, GroceryWalletTest.kt
FR-06	Накопительный счёт и вклад	Частично	Операции реализованы в data_storage.kt и UI в front/indicate.kt. Начисление процентов/завершение вклада находится в onSleep(), но вызов onSleep() из UI отсутствует.
FR-07	Магазин с редактируемым ассортиментом	Готово	MarketCatalog.kt, MarketDialog.kt
FR-08	Покупка одежды и шкаф с экипировкой	Готово	wardrobe/, PlayerStateTest.kt, GameIntegrationTest.kt
FR-09	Мини-игра склада	Готово	games/warehouse/, BlockBlastRulesTest.kt
FR-10	Велогонка	Готово	games/bikes/, BikeRulesTest.kt
FR-11	Продуктовая мини-игра с бюджетом	Готово	games/grocery/, RunnerRulesTest.kt, GroceryWalletTest.kt
FR-12	Локальное сохранение прогресса	Готово	PlayerSave.kt, GameDataStore, PlayerPersistenceTest.kt

ID	Требование	Статус	Реализация / проверка
FR-13	Состояние и рост питомца	Готово	В прототипе роль питомца выполняет основной игровой персонаж. Его состояние задаётся показателями food и weariness, визуальное развитие — одеждой и экипировкой, а общий рост отражается через игровой прогресс, экономику и открываемый контент. Отдельная сущность Pet в текущей архитектуре не требуется.
FR-14	Профиль пользователя и удаление профиля внутри приложения	Не реализовано	Персональный профиль отсутствует; удаление локальных данных выполняется средствами Android.
FR-15	Серверная синхронизация / API	Не применяется	Backend и сетевой API отсутствуют.

7. Формулы и правила экономики, наград и состояния

Базовый кошелёк

Покупка товара цены p выполняется только если ID валиден, товар ещё не куплен и $money \geq p$:

$$money' = money - p$$

В остальных случаях баланс и инвентарь не меняются.

Накопительный счёт

Пополнение:

$$money' = money - amount$$

$$savings' = savings + amount$$

Снятие — обратная операция, если средств достаточно.

В `onSleep()` реализовано начисление 1% на текущий остаток:

$$interest = \text{floor}(savings \times 0.01)$$

$$savings' = savings + interest$$

При последовательных вызовах это даёт капитализацию, поскольку каждый следующий процент считается от уже увеличенного остатка. В текущем UI `onSleep()` не вызывается, поэтому механика начисления пока фактически не запускается.

Вклад

При открытии:

```
depositBalance = amount
depositDaysLeft = 2
depositActive   = true
money'          = money - amount
```

На каждом `onSleep()` срок уменьшается на 1. При достижении нуля:

```
finalAmount = floor(depositBalance × 1.022)
money' = money + finalAmount
```

То есть прототип использует доходность 2.2% за игровой срок в два сна. Досрочное снятие в текущей версии не реализовано.

Состояние персонажа

Все прямые изменения ограничиваются диапазоном:

```
food      = clamp(food + Δfood, 0, 100)
weariness = clamp(weariness + Δweariness, 0, 100)
money     = max(0, money + Δmoney)
```

Сон:

```
food'      = clamp(food - 10, 0, 100)
weariness' = clamp(weariness + 30, 0, 100)
```

Название `weariness` семантически не совпадает с поведением шкалы: высокое значение отображается зелёным и сон увеличивает его. Фактически показатель ведёт себя как «бодрость/энергия».

Склад

Размер поля по умолчанию — 8×8, целевой счёт — 500, максимальная награда — 100.

```
очки за размещение = число клеток фигуры
очки за очистку     = 10 × lines2 × combo
successRate         = clamp(score / 500, 0, 1)
reward              = floor(100 × max(0.5, successRate))
```

Следовательно, при корректном завершении награда склада находится в диапазоне 50..100 монет.

Велогонка

Для `place` — места игрока, `racers` — числа участников, `collisions` — количества столкновений:

```
successRate = clamp(
    1 - 0.7 × (place - 1) / max(1, racers - 1) - 0.08 × collisions,
    0.1,
    1.0
)

reward = clamp(50 - 10 × (place - 1) - 5 × collisions, 10, 50)
```

Продуктовая мини-игра

Для каждого вида товара задана базовая цена. Обычная цена генерируется примерно в диапазоне 60–130% базовой и округляется вверх до кратного 5. Дешёвым считается товар с ценой не выше 85% базы. Испорченный товар получает дополнительный коэффициент 0.5.

```
completion = collectedNeeded / totalNeeded
thrift      = clamp(optimalCost / spent, 0, 1), если spent > 0, иначе 0
successRate = clamp(completion × thrift × crashPenalty, 0, 1)
crashPenalty = 0.5 при столкновении, иначе 1.0
```

Деньги за подобранный товар списываются сразу. Итоговая мини-игра не начисляет награду и повторно чек не списывает.

Питомец

В прототипе питомец реализован через основного игрового персонажа. Поэтому его состояние рассчитывается теми же правилами, что и состояние персонажа: показатели food и weariness изменяются игровыми действиями и ограничиваются диапазоном 0..100. Рост трактуется как совокупный игровой прогресс: изменение состояния, накопление и расход средств, открытие учебного контента, покупка и экипировка предметов. Отдельная числовая шкала уровня или эволюции питомца не используется — это сознательное упрощение прототипа.

8. Карта образовательного контента

Тема	Ожидаемый навык	Сценарий в приложении	Правильная логика и объяснение для ребёнка
Обязательные / необязательные траты	Отличать потребность от желания	Покупка продуктов по списку	Сначала оплачиваются необходимые вещи; необязательные покупки допустимы только из оставшегося бюджета.
Накопления	Откладывать часть денег на цель	Пролог: сдачу можно не тратить	Накопление — это отложенное потребление: часть денег сохраняется ради более важной будущей цели.
Дебетовая карта	Понимать, что карта расходует собственные деньги	Пролог и получение карты	Дебетовая карта — доступ к собственному банковскому счёту; потратить больше остатка нельзя.
Детская банковская карта	Понимать отличие детской карты от личного счёта	Пролог	Детская карта обычно связана со счётом/контролем родителей; самостоятельность ограничена настройками взрослых.
Банковский счёт	Отличать счёт от пластиковой карты	Диалог в банке	Деньги хранятся на счёте, карта лишь даёт к нему доступ; потеря карты не означает потерю счёта.

Тема	Ожидаемый навык	Сценарий в приложении	Правильная логика и объяснение для ребёнка
Накопительный счёт	Сравнивать ликвидность и доходность	Банковское окно	Деньги можно пополнять и снимать, а проценты начисляются на остаток. В прототипе заложено +1% за игровой «сон».
Вклад	Понимать обмен ликвидности на фиксированный срок/доход	Банковское окно	Деньги размещаются на срок; в прототипе срок равен двум игровым снам, итог — 102.2% внесённой суммы.
Процентная ставка	Читать процент как цену/доход за использование денег	Диалог с сотрудником банка	Процент показывает, сколько добавится к сумме или сколько придётся доплатить по долгу.
Кредит	Понимать возврат долга с процентами	Диалог в банке	Кредит — не дополнительные бесплатные деньги: нужно вернуть основную сумму и проценты.
Банк	Понимать базовую роль банка	Диалог в банке	Банк хранит деньги, проводит платежи, принимает вклады и выдаёт кредиты; одна из моделей дохода — разница процентных ставок.
НДФЛ	Понимать удержание налога из дохода	Задание, связанное с работой	Зарплата «на руки» может быть меньше начисленной, потому что часть дохода удерживается как налог.
НДС	Понимать налог внутри цены товара	Задание, связанное с магазином	НДС уже включён в цену большинства товаров: покупатель видит конечную цену и платит налог как её часть.
Сложный процент	Понимать начисление процентов на проценты	Диалог в банке; потенциально накопительный счёт	При капитализации новая база включает ранее начисленный процент, поэтому рост ускоряется.
Пассивный доход	Отличать доход от капитала от оплаты труда	Диалог в банке	Например, проценты по вкладу возникают из размещённых денег, а не из оплаты конкретного часа работы.
Инфляция	Понимать снижение покупательной способности денег	Диалог в банке	Если цены растут, та же сумма со временем покупает меньше товаров.
Валютный курс	Понимать цену одной валюты в другой	Термин присутствует в словаре	Курс показывает, сколько единиц одной валюты нужно для покупки единицы другой. Отдельный игровой сценарий пока отсутствует.

Тема	Ожидаемый навык	Сценарий в приложении	Правильная логика и объяснение для ребёнка
Экономный выбор в магазине	Сравнивать цену, качество и бюджет	Grocery runner	Самый дешёвый вариант не всегда лучший: испорченный товар может быть дешевле, но приводит к потере денег и ошибке.
Доход за результат	Связывать результат работы и вознаграждение	Склад и велогонка	Игрок получает награду по формуле, зависящей от качества результата; повторное завершение одной сессии не дублирует выплату.

9. UX/UI-решения и доступность

Обоснование UX/UI

- **Постоянный верхний HUD** показывает деньги, сытость и состояние персонажа, чтобы игрок видел последствия решений без перехода на отдельный экран.
- **Нижняя навигация** хранит основные действия в фиксированном месте: карта, банк, термины/квесты и магазин.
- **Миникарта** разделяет мир на понятные бытовые контексты: дом, банк, парк, склад, магазин, школа.
- **Контекстный запуск мини-игр** выполняется из соответствующей локации, а не из абстрактного меню: работа — на складе, покупки — в магазине, гонка — в парке.
- **Магазин строится из данных MarketCatalog**, поэтому ассортимент можно менять без переписывания UI карточек.
- **Шкаф показывает только купленные вещи**, что делает причинно-следственную связь «заработал → купил → использовал» явной.
- **Финансовые понятия раскрываются через сюжет и действие**, а словарь остаётся справочным экраном для повторения.

Реализованные механизмы доступности

- ключевым интерактивным зонам карты и переходам заданы `contentDescription`;
- Compose использует размеры текста в `sp`, поэтому системное масштабирование шрифта частично поддерживается платформой;
- основной экран использует `safeDrawingPadding()`, не размещая критичные элементы под системными панелями;
- основные действия представлены Material-компонентами с крупными зонами нажатия;
- важные действия дополнительно покрыты Compose UI-тестами и `semantic/test tags`.

Настройки доступности

Отдельного экрана настроек доступности в текущей версии **нет**. Не реализованы пользовательские переключатели высокого контраста, увеличенного интерфейса, уменьшения анимации, режима для дальтонизма и отдельной настройки озвучивания.

Для следующей версии рекомендуется добавить:

1. режим повышенного контраста;

2. увеличение UI независимо от системного font scale;
 3. уменьшение/отключение анимаций;
 4. дублирование цветовых индикаторов текстом/иконографикой;
 5. полный TalkBack-аудит всех кликабельных изображений;
 6. проверку интерфейса при font scale 1.3–2.0×.
-

10. Android permissions, собираемые данные и удаление профиля

Разрешения

В AndroidManifest.xml нет объявленных uses-permission. Приложение не запрашивает камеру, микрофон, контакты, местоположение, уведомления или доступ к сети.

Состав данных

Текущая версия хранит на устройстве только игровые данные:

- баланс и состояние персонажа;
- остатки накопительного счёта и вклада;
- купленные и надетые товары;
- идентификатор последней засчитанной мини-игровой сессии;
- завершение пролога;
- открытые термины, диалоги и задания.

Персональные данные пользователя, e-mail, номер телефона, геолокация и идентификаторы рекламных систем в коде не собираются.

Локальные хранилища

- SharedPreferences: player_state;
- Preferences DataStore: game_progress.

android:allowBackup="true"; файлы backup_rules.xml и data_extraction_rules.xml не содержат явных исключений. Поэтому приложение потенциально допускает стандартное резервное копирование Android. Если по требованиям проекта данные должны оставаться строго локальными и не восстанавливаться из cloud backup, правила резервного копирования нужно ужесточить перед релизом.

Удаление данных

Встроенной кнопки «Удалить профиль» сейчас нет. Локальные данные удаляются стандартными средствами Android:

Настройки → Приложения → My Application → Хранилище → Очистить данные

Для тестовой/разработческой сборки:

```
adb shell pm clear com.example.myapplication
```

Удаление приложения также очищает локальные данные, но при включённом Android Backup часть состояния потенциально может восстановиться после повторной установки. Для полного

соответствия требованию об удалении профиля рекомендуется добавить действие «Сбросить прогресс» внутри приложения и определить явную политику backup/restore.

11. Тест-кейсы и отчёт о проверке

Автоматические проверки, найденные в репозитории

Архив содержит результаты testDebugUnitTest: **45 тестов, 0 failures, 0 errors**.

Покрываются:

- защита от повторной покупки и двойного списания;
- запрет экипировки некупленной вещи и один предмет на слот;
- миграция старых ID инвентаря;
- однократное начисление награды за одну сессию;
- невозможность уйти в отрицательный бюджет продуктовой игры;
- формулы цен, экономности и успеха grocery;
- правила фигур и очистки линий склада;
- расчёт награды склада;
- правила места, столкновений и награды велогонки;
- валидность ID/спрайтов.

Сохранённый connected-test отчёт от 2026-09-29 содержит **2/2 passed** теста PrologTest на Pixel_4 (AVD) - 17: полный первый пролог и пропуск пролога после сохранения.

В исходниках также есть GameIntegrationTest.kt и PlayerPersistenceTest.kt, однако их успешный запуск не зафиксирован в сохранённом connected-test XML текущего архива.

Рекомендуемый smoke-test на физическом устройстве

ТС	Проверка	Ожидаемый результат
ТС-01	Чистая установка и первый запуск	Отображается пролог; после полного прохождения открывается детская.
ТС-02	Повторный запуск	Пролог не показывается повторно.
ТС-03	Дом: детская → гостиная → кухня → детская	Переходы работают, UI не перекрывается системными панелями.
ТС-04	Миникарта: дом, парк, склад, магазин, банк	Каждая реализованная локация открывается; школа должна быть отмечена как известное ограничение.
ТС-05	Купить одежду → открыть шкаф → надеть	Баланс уменьшается один раз, вещь появляется в шкафу и на персонаже.
ТС-06	Повторно купить тот же товар	Повторного списания нет.
ТС-07	Склад	Мини-игра запускается, завершается, награда поступает в общий кошелек один раз.
ТС-08	Велогонка	Игра запускается; место и столкновения влияют на награду.

ТС	Проверка	Ожидаемый результат
ТС-09	Продуктовый магазин	Подбор товара немедленно уменьшает баланс; уйти в минус нельзя.
ТС-10	Поворот экрана во время активной мини-игры	Поддерживаемая активная сессия не дублирует награду/покупки.
ТС-11	Заккрыть и снова открыть приложение после покупок	Баланс, покупки и экипировка восстанавливаются.
ТС-12	Очистить данные приложения	Прогресс и экономика сбрасываются к начальному состоянию.

Краткий отчёт по физическому устройству

В текущем архиве нет machine-readable отчёта или протокола, подтверждающего полный прогон именно на физическом устройстве. Этот пункт нельзя корректно отметить как PASS без фактической проверки.

Перед сдачей заполнить:

Устройство: _____
 Версия Android: _____
 Версия APK / commit: _____
 Дата проверки: _____
 Результат ТС-01...ТС-12: _____
 Критические дефекты: _____

12. Известные ограничения и план развития

Известные ограничения прототипа

- Школа не подключена к маршрутизации.** Миникарта переводит состояние в school, но GameApp не имеет отдельной ветки "school" и попадает в fallback.
- Квесты не завершаются автоматически.** Структура и UI есть, но текущие DialogResult не выставляют unlockQuest.
- Банковский игровой цикл неполный.** onSleep() содержит проценты и завершение вклада, но нигде не вызывается из UI; кнопка снятия для вклада фактически ничего не делает и не объясняет причину пользователю.
- Товары вне одежды не имеют игрового эффекта. Мебель, спорт и техника покупаются и сохраняются как ID, но не изменяют комнаты/механику.
- Кэшбэк и аналитика упоминаются в тексте пролога, но отдельная механика не реализована.
- Профиль пользователя отсутствует. Нет имени, аватара-профиля, нескольких слотов сохранения или облачной синхронизации.
- Отчёт о проверке именно на физическом устройстве не сохранён в репозитории; перед сдачей требуется зафиксировать модель устройства, версию Android и результаты smoke-test.
- Нет встроенного удаления прогресса и явной политики backup.**
- Release signing не настроен в Gradle.** Подписанный APK сейчас формируется вручную через Android Studio/CI.
- Лицензии графических ассетов не задокументированы в репозитории.**

11. **Часть исходников содержит черновой/тестовый текст и legacy-код, который следует удалить перед публичным релизом.**
12. **Строки интерфейса в основном захардкожены в Kotlin**, локализация через strings.xml практически не используется.

План развития

P0 — перед сдачей/релизом: подключить школу или убрать активную зону; связать квесты с игровыми событиями; подключить цикл сна к банковской механике; отключить/объяснить досрочное снятие вклада; удалить черновой текст; провести полный физический smoke-test; оформить лицензии ассетов; настроить release signing.

P1 — функциональное развитие: реализовать эффект мебели/техники/спорта, кэшбэк и аналитику расходов; добавить профиль/сброс прогресса; при необходимости расширить визуальные стадии развития персонажа без изменения базовой модели питомца-персонажа.

P2 — качество: вынести пользовательские строки в resources, добавить локализацию, TalkBack-аудит, accessibility settings, скриншотные/регрессионные UI-тесты и CI pipeline.

13. Сторонние библиотеки и лицензии

Runtime / AndroidX

Компонент	Версия	Назначение	Лицензия
AndroidX Core KTX	1.10.1	Kotlin extensions Android API	Apache License 2.0
AndroidX Activity Compose	1.8.0	Compose Activity integration	Apache License 2.0
AndroidX Lifecycle Runtime KTX	2.6.1	lifecycle runtime	Apache License 2.0
Jetpack Compose UI / Material3	BOM 2026.02.01	UI framework	Apache License 2.0
AndroidX DataStore Preferences	1.1.1	сохранение game_progress	Apache License 2.0
Kotlin	plugin 2.2.10	язык/Compose plugin	Apache License 2.0

Test dependencies

Компонент	Версия	Лицензия
JUnit 4	4.13.2	Eclipse Public License 1.0
Kotlin Test JUnit	1.9.24	Apache License 2.0
AndroidX Test JUnit	1.3.0	Apache License 2.0
Espresso Core	3.7.0	Apache License 2.0
Compose UI Test	версия через Compose BOM	Apache License 2.0

Build tooling

Gradle Wrapper 9.5.0, Android Gradle Plugin 9.3.2 и Foojay toolchain resolver используются только при сборке и не являются пользовательским контентом приложения. Перед публикацией

артефактов следует сохранить стандартные NOTICE/LICENSE требования соответствующих проектов, если того требует способ распространения исходного кода.

Шрифты

Отдельные файлы шрифтов в `app/src/main/res/font/` отсутствуют. Используются системные/Material-шрифты платформы.

Изображения

Графические ресурсы находятся в:

- `app/src/main/res/drawable/;`
- `app/src/main/res/drawable-nodpi/;`
- исходных каталогах `Clothes/` и `room/`.

В репозитории **нет файла LICENSE/NOTICE или таблицы атрибуции, позволяющей определить автора и лицензию каждого изображения**. До публичного релиза необходимо для каждого внешнего ассета зафиксировать: имя файла, автора/источник, ссылку на источник, тип лицензии и требуемую атрибуцию. Ассеты неизвестного происхождения нельзя автоматически считать свободными для коммерческого распространения.

Звуки

Аудиофайлы в `res/raw` или иных каталогах проекта не обнаружены. Отдельных звуковых ассетов в текущей версии нет.

14. Команды проверки перед сдачей

```
# Unit tests
.\gradlew.bat :app:testDebugUnitTest

# Instrumented tests — нужен подключённый эмулятор/телефон
.\gradlew.bat :app:connectedDebugAndroidTest

# Debug APK
.\gradlew.bat :app:assembleDebug

# Release APK (unsigned без signingConfig)
.\gradlew.bat :app:assembleRelease
```

Минимальный release gate:

- unit tests проходят;
- instrumented tests проходят на целевой версии Android;
- выполнен физический smoke-test;
- подписанный release APK устанавливается с чистого состояния;
- школа/квесты/банк либо исправлены, либо явно исключены из заявленного объёма;
- лицензии всех изображений подтверждены;
- из релизной ветки удалены `local.properties`, `build outputs`, временные файлы и секреты `signing`.